

# S06 catalog publication and recovery

Internal Rust storage feature. No weather route, provider request, production resource or deployment is enabled by this task. The example uses a fresh local test S3 peer with obvious dummy credentials; it does not verify AWS SigV4, IAM, Versioning, lifecycle, response parity or AWS latency.

## Run the actual HTTP scenario

From server2/, with the configured Rust toolchain and Python3:

```
cargo build --release --example catalogs_smoke
python3 tests/storage/catalog_peer.py --smoke target/release/examples/catalogs_smoke
```

The public peer starts an ephemeral IPv4 loopback port, runs the release example, and closes its server/child resources. Both the reusable fault peer and release scenario are maintained here. The captured usage image shows actual stdout from this command, not a simulated output. Detailed Rust wire assertions run through `cargo test --workspace`; the example is a separate functional execution.

## Cold read and concurrent writes

Prepare a complete `GroupDeclaration` before publishing any raw body. Members contain canonical S05 envelopes with `fetch_group:None`, ordered pages and exact sorted catalog ownership. Hash the declaration, ordered members and partitions; attach its resulting immutable reference before each raw PUT. Store the descriptor first, then all raw bodies, then union each catalog using opaque ETag/If-Match. Only all-sibling and exact raw verification returns a checked complete acquisition. A raw PUT acknowledgment is body durability only.

Catalog keys are `index/v2/{source}/{kind}/{key_sha256}/{partition}.json`. Descriptors use `index/v2/groups/{group_sha256}.json`. JSON stores identity/envelope metadata only. Every revision remains; arbitrary normalized values are rejected by typed schemas. Group SHA has no self-reference because declared envelopes exclude group refs. Pages 1..N must be complete, unique and ordered. Previously ungrouped/differently grouped raw identities cannot be retroactively affiliated.

Whole-history Lookup pins each involved catalog once. `CompleteSet` exposes complete acquisitions and every dependency identity, including absent siblings through optional generation/ETag tokens. `A/new` plus `B/healthy-old` excludes the new group. Old complete acquisitions may remain usable with `excluded_groups()>0`; consumers must schedule re-lookup on exclusions, including missing descriptor/body cases, without assuming unchanged target ETag proves complete coverage. S07 owns this cache refresh/invalidation policy. Earliest/latest, absent-only top-level field merge and whole-list replacement are derived in memory from ordered complete acquisitions; present null/zero/false fields are never overwritten by absent merge.

## Bounded history reads and publication scope

Archival catalogs retain every revision. Normal read-budget exhaustion is Capacity or Timeout, not corruption and not a provider miss. Full Lookup is the only `FullHistory CompleteSet`; only that coverage permits fold. Publication verifies only the newly declared group and returns `PublishedGroup` coverage; attempting to fold that scoped result as history is rejected. Parse its explicit records directly when the desired operation is that single acquisition.

Use `lookup_page(id, order, cursor, maximum_acquisitions, deadline)` for bounded whole-acquisition pages in canonical target-catalog entry order. Pages have `Paged` coverage and no fold method. Ordered provider pages remain together. The opaque private cursor pins target generation/ETag/version and prior sibling dependencies; cross-ID/order use is invalid and changed tokens yield `Incomplete/restart`. Consumers must not merge one page as all history. Collect every page and revalidate dependencies before a cross-page historical result; growing dependency checks may still exceed the deadline or cursor metadata cap, requiring safe fallback.

`lookup_acquisition(id, exact_member_identity, deadline)` is the independent escape hatch: verify target membership and that member's whole immutable group/siblings. It returns `Targeted` coverage with no continuation, which does not mean exhausted history. Old/new acquisitions remain accessible within their own group/catalog bounds without reading unrelated historical bodies. Sequential reads and S07/S09 gates remain.

Preflight rejects an oversized new group with Capacity before any PUT. It checks raw total, descriptor, projected catalog union and conservative final metadata: descriptor bytes + partition count × control-document limit must fit the retained control cap.

Format allows 16 partitions, but default admission permits at most 7 full-size catalogs plus descriptor; smaller store caps may admit fewer. No archive pruning is used. Preflight buffers are dropped before CAS, and completed CAS-phase buffers before fresh new-group pinning. Control accounting is bounded per retained phase, not cumulative wire bytes across all phases; CAS work and the outer deadline still bound retries. Concurrent growth or uncertain sent writes may remain Ambiguous.

## Repair and coverage

Call `repair` with an explicit `RepairScope` containing the target catalog and all candidate canonical raw periods/slots for that provider resolution. It scans even healthy old catalogs, follows every LIST page, verifies raw key/hash/envelope and full group descriptor/membership, then CAS repairs all required catalogs and re-pins siblings. A partial LIST, malformed XML/token, missing/corrupt member, wrong raw hash, deadline or cap never becomes a complete empty response.

`LookupOutcome::Incomplete` is not provider admission. `RepairOutcome::CompleteEmpty` proves absence only in the caller-supplied finite scope after an exhausted clean scan; it does not prove global archive absence. `Complete` contains verified data; `Indexed {scope, dependencies}` records an exhausted, verified nonempty scan and successful identity-index repair when full-history materialization exceeds capacity. It is neither a serving `CompleteSet` nor `CompleteEmpty` and cannot authorize providers; read checked pages or targeted acquisitions afterward. Incomplete or errors leave valid memory/legacy error fallback. Discovery retains envelopes only; verified body buffers are released before reloading each complete group, without double-counting discarded bodies as retained output. No provider calls or budget reservations occur in this utility.

The caller must map every supplied period to the target partition deterministically. Foreign-owned grouped records are Incomplete; ungrouped singletons rely on this caller ownership assertion. This utility cannot infer a geocode/calendar layout. Default scope has up to 16 exact canonical periods/slots. Later geocode integration must supply its approved month-based/projection identities and prove 30-day validity plus multi-month restore (including January 31 to March 1); S06 has not executed that geocode fixture. Calendar/coverage policy and provider parsers remain later work. A descriptor with no discoverable raw body cannot be found by raw-prefix scan alone; known indexed missing bodies remain incomplete. No claim of global orphan absence is made from direct catalogs or finite scans.

## Failures and limits

Completed conditional PUT 200 is the only write acknowledgment. HTTP 412 causes re-read/union; HTTP 409 and unknown writes reconcile within a bounded attempt count. A sent unknown PUT remains uncertain through a later read 404/retry rejection: return Ambiguous unless exact stored identity/union proves success. A first direct 403 stays `Status(403)`; malformed identities/corruption fail closed. Cancellation or an operation deadline after publication begins cannot return a complete group. No hidden SDK retry or third provider acquisition is used.

Defaults: JSON/XML 1 MiB, catalog 2,048 entries, group 128 members, 16 partitions, 8 LIST pages per prefix, 2,048 raw candidates, 1,000 keys/page, 4 CAS attempts, 8 MiB serialized control budget per retained phase and 32 MiB decoded serving output. Repair discovery discards raw bodies, and each group reload/output is bounded separately. Legal chunked or close-delimited control responses may omit Content-Length; streaming caps remain enforced and any declared length must match. Empty truncated LIST pages follow valid opaque tokens under existing page and repeated-token bounds. The caller's absolute monotonic deadline is capped by the store's 3 s default; all credential waits, siblings, raw reads, CPU work and CAS/LIST count toward it. Hard setter ceilings fail safely. Bounded I/O operation admission and CPU workers remain held for started parsing work after cancellation. Within one lookup/repair, descriptor and raw-member GETs currently run sequentially. The I/O limit of 16 bounds concurrent operations; it does not produce 16 parallel GETs inside one lookup. The architecture cold-latency model assumes prospective bounded fan-out, not this implementation. S07 integration and S09 feasibility/cutover measurements must resolve this limit before a cold weather route is switched. No AWS latency or complete-day timing is established by the small local smoke. Limits are per store/operation, not hard process RSS; caller input/results and concurrent stores consume memory.

Real AWS role 404 versus 403, signature/header behavior, Versioning/no-delete policy and performance remain staging gates. No delete/lifecycle/pack work is implemented. Catalog current revisions are retained; version-LIST is unnecessary on this path. Oversized catalogs fail rather than silently pruning revisions. Route cutover and cold-cell parity/fallback gates remain unchanged.

## Verification artifacts

The editable manual and rendered PDF share this content. The selected usage capture and hash manifest live under docs/evidence/tasks/server2-catalogs/. The release scenario checks cold complete pages, A/new+B/old exclusion, healthy multi-page repair, committed response loss, delayed first commit with retry403, first direct403, bounded ordered pages/targeted scope/fold guard, chunked LIST and empty truncated continuation. See the [architecture publication contract](#) and [publication sequence](#).

## Actual local CLI usage

```
S06 - actual release HTTP smoke | loopback only

$ python3 tests/storage/catalog_peer.py --smoke target/release/examples/catalogs_smoke

publish: complete group, 2 ordered raw pages, 2 catalog partitions
cold: 1 checked acquisition, 2 pinned dependencies, providers disabled
partial: A/new + B/healthy-old excluded; preceding complete group retained
repair: healthy catalog + paginated LIST restored every revision
response loss: committed CAS reconciled from exact union
late commit: lost first PUT + read404 + retry403 remains Ambiguous; exact catalog exists
direct rejection: first PUT403 retained, no group success
bounded read: whole 2-page acquisition, opaque continuation and exact targeted scope; scoped publication
fold blocked
streamed repair: chunked LIST and empty truncated continuation exhaust only the explicit empty scope
PASS local real HTTP smoke (141.31 ms); not AWS latency/auth or API parity
```